



fit@hcmus

PHÁT TRIỂN PHẦN MỀM CHO THIẾT BỊ DI ĐỘNG

Chủ đề: Kotlin Coroutine

Nhóm 4: Lobotomy Corporation

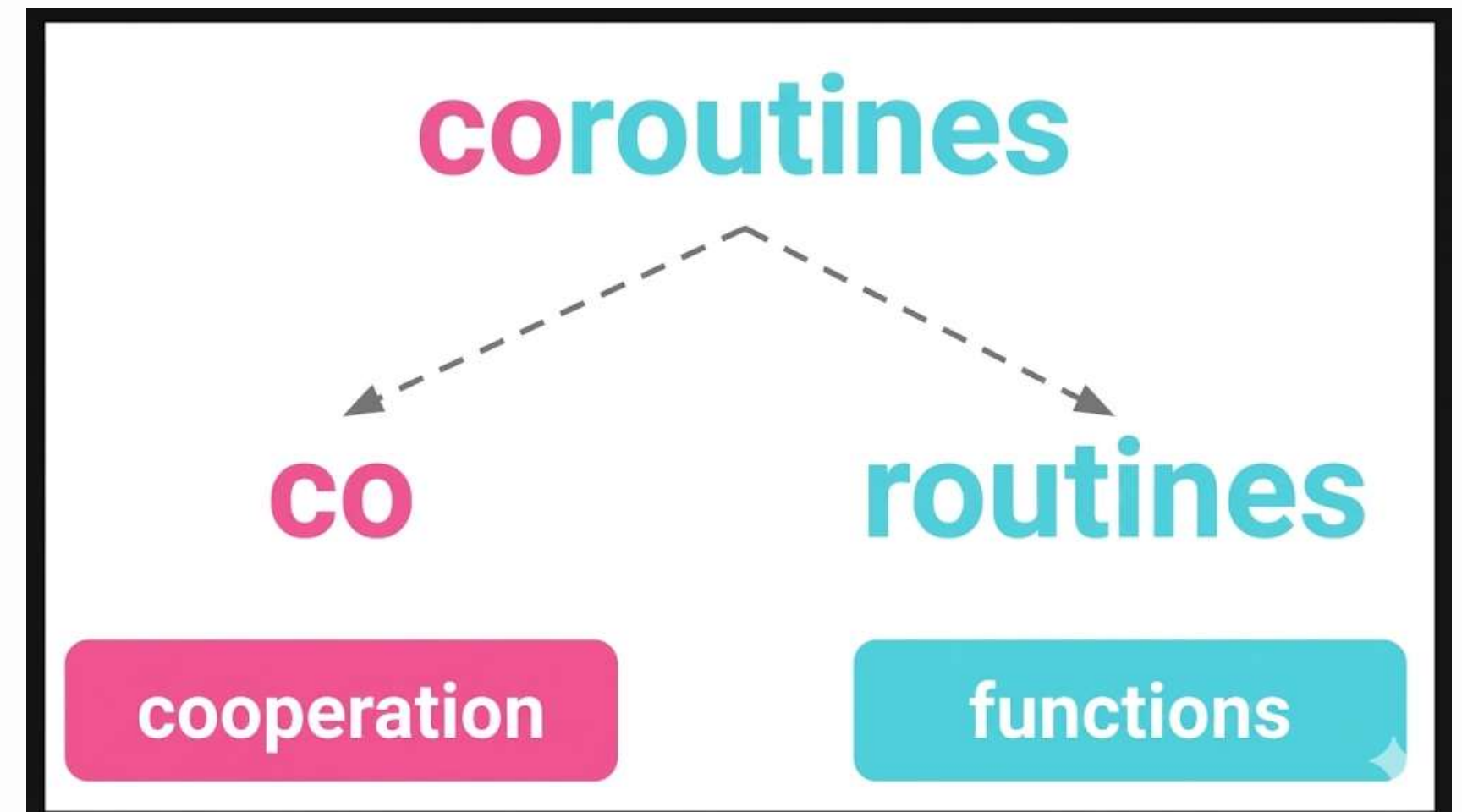


Nội dung

- ▶ Tổng quan về Coroutines Kotlin
- ▶ Coroutine Basics
- ▶ Cancellation
- ▶ Asynchronous Flow

Tổng quan về Coroutines Kotlin

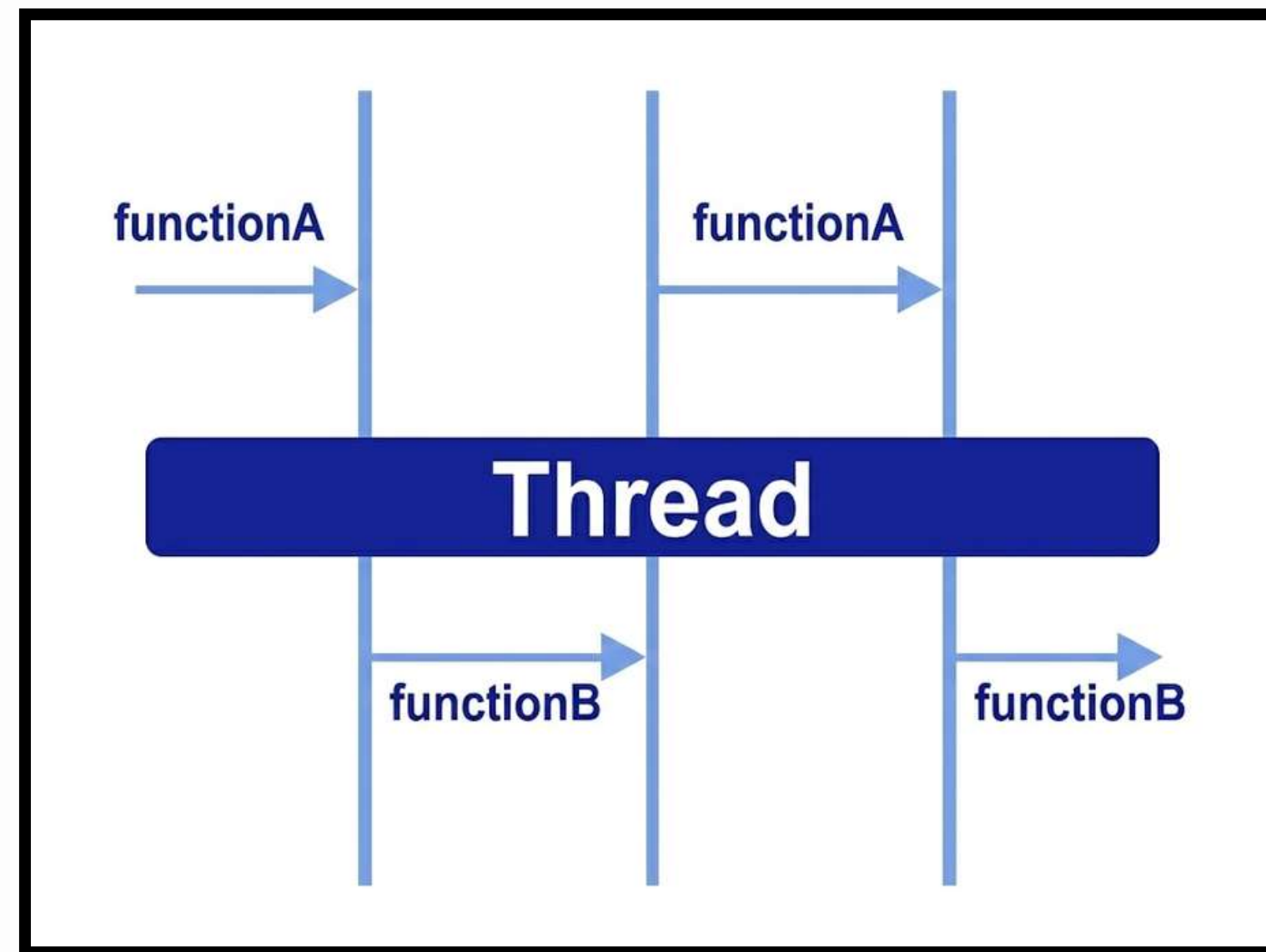
Các ứng dụng thường cần thực hiện nhiều tác vụ cùng một lúc, chẳng hạn như phản hồi đầu vào của người dùng, tải dữ liệu hoặc cập nhật màn hình.



Hình 1: Coroutines
Kotlin

Tổng quan về Coroutines Kotlin

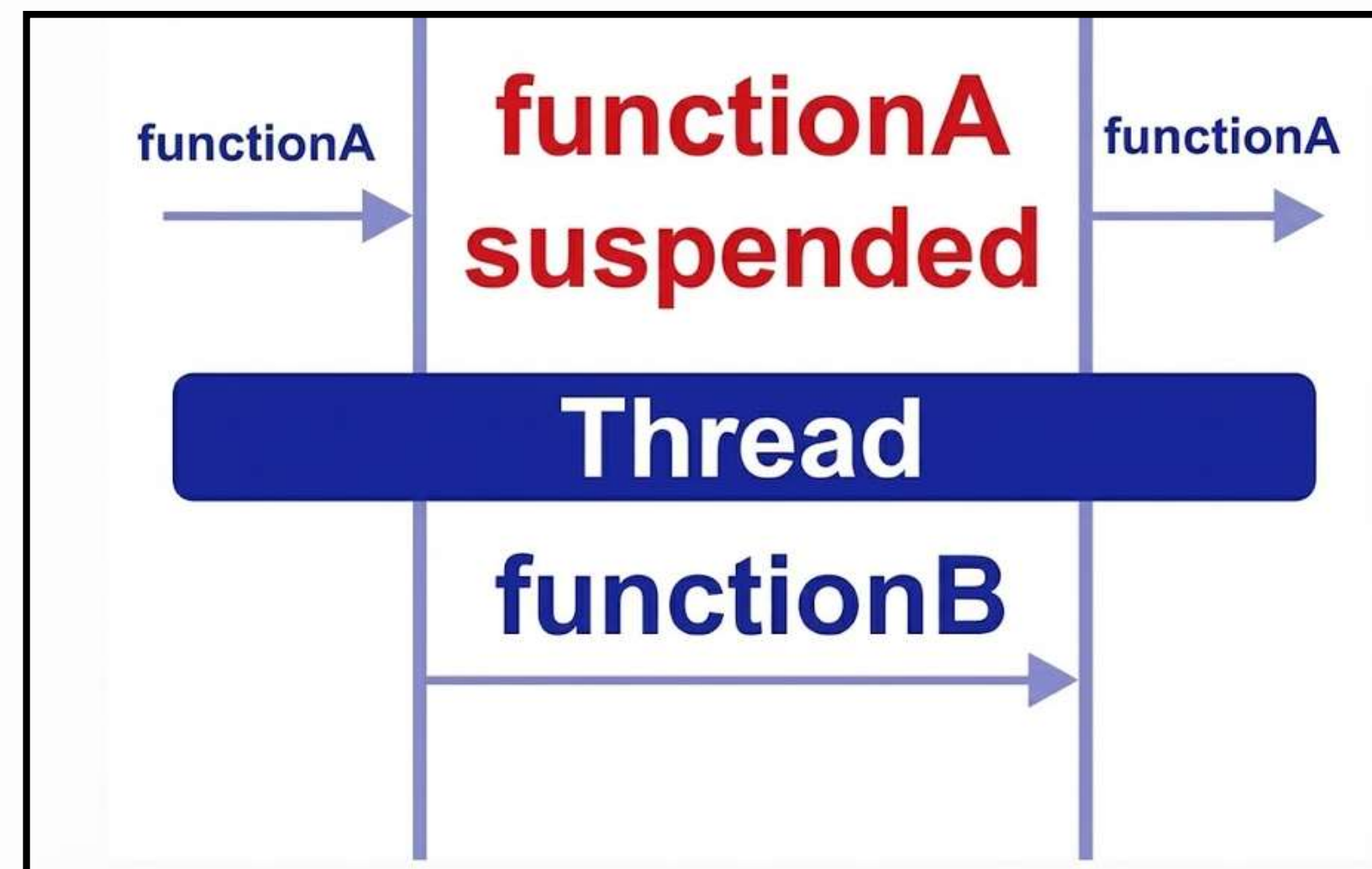
Cách phổ biến nhất để chạy các tác vụ đồng thời là sử dụng luồng (thread), là các đường dẫn thực thi độc lập được quản lý bởi hệ điều hành.



Hình 2:
Thread

Tổng quan về Coroutines Kotlin

Kotlin sử dụng lập trình bất đồng bộ được xây dựng xung quanh coroutine, cho phép viết mã bất đồng bộ theo phong cách tự nhiên, tuần tự bằng cách sử dụng các hàm tạm dừng (suspend functions).



Hình 3: Suspend function

Coroutine

Basics

01

Suspending function

02

Coroutine scope and structured concurrency

03

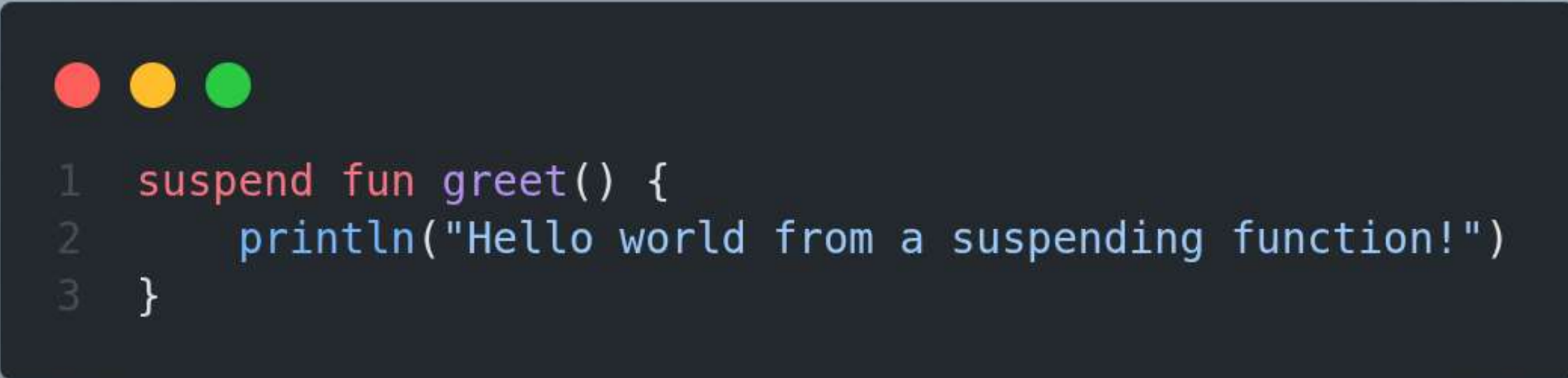
Coroutine builder function

04

Coroutine dispatchers

Suspending function

- **Hàm Tạm dừng (Suspending Function)** là hàm có khả năng tạm dừng (suspend) việc thực thi của nó và tiếp tục (resume) sau này, mà không làm chặn (block) luồng (thread) đang chạy.
- **Cú pháp:** Được khai báo bằng từ khóa suspend



```
1 suspend fun greet() {  
2     println("Hello world from a suspending function!")  
3 }
```

Hình 4: Cú pháp khai báo suspending function

Suspending function

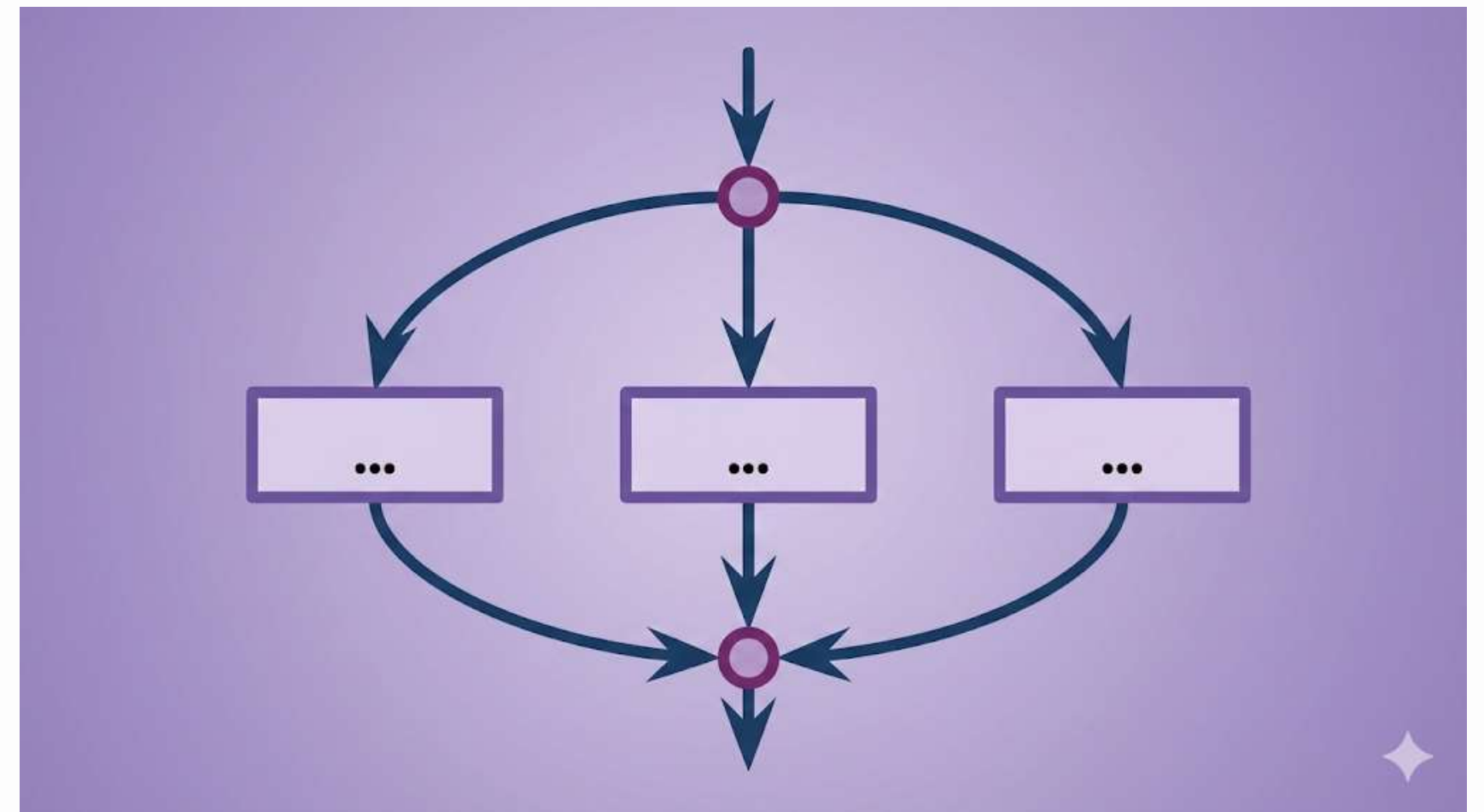
Quy tắc gọi: một suspending function chỉ có thể được gọi từ một suspending function khác hoặc từ một Coroutine.

```
1 suspend fun main() {  
2     showUserInfo()  
3 }  
4  
5 suspend fun showUserInfo() {  
6     println("Loading user...")  
7     greet()  
8     println("User: John Smith")  
9 }  
10  
11 suspend fun greet() {  
12     println("Hello world from a suspending function")  
13 }
```

Hình 5: Ví dụ về chuỗi gọi suspending function

Coroutine scope and structured concurrency

Khi ứng dụng chạy nhiều Coroutines, việc quản lý vòng đời và xử lý lỗi của chúng trở nên phức tạp. Kotlin Coroutines giải quyết điều này bằng cách áp dụng nguyên tắc Structured Concurrency.



Hình 6: Structured concurrency

Coroutine scope and structured concurrency

Cấu trúc cây: Coroutines được tổ chức thành cấu trúc cha-con.

Liên kết Vòng đời: Vòng đời của các Coroutines này được liên kết chặt chẽ:

- Một Coroutine cha sẽ chờ tất cả Coroutine con hoàn thành trước khi kết thúc.
- Nếu Coroutine cha bị hủy hoặc gặp lỗi, toàn bộ Coroutine con của nó cũng sẽ bị hủy theo.

Coroutine scope and structured concurrency

```
1 suspend fun main() {  
2     // Root of the coroutine subtree  
3     coroutineScope { // this: CoroutineScope  
4         this.launch {  
5             this.launch {  
6                 delay(2.seconds)  
7                 println("Child of the enclosing coroutine completed")  
8             }  
9             println("Child coroutine 1 completed")  
10        }  
11        this.launch {  
12            delay(1.seconds)  
13            println("Child coroutine 2 completed")  
14        }  
15    }  
16    // Runs only after all children in the coroutineScope have completed  
17    println("Coroutine scope completed")  
18 }
```

Hình 7: Ví dụ về Coroutine scope

Coroutine builder function

Coroutine builder function là các hàm nhận vào một suspend lambda để định nghĩa một coroutine cần chạy. Các builder này luôn cần một CoroutineScope để hoạt động.

Một số coroutine builder phổ biến gồm:

- CoroutineScope.launch()
- CoroutineScope.async()
- runBlocking()

Coroutine builder function

01

CoroutineScope.launch()

```
1 suspend fun performBackgroundWork() = coroutineScope {  
2     // this: CoroutineScope  
3     // Starts a coroutine that runs without blocking the scope  
4     this.launch {  
5         // Suspends to simulate background work  
6         delay(100.milliseconds)  
7         println("Sending notification in background")  
8     }  
9  
10    // Main coroutine continues while a previous one suspends  
11    println("Scope continues")  
12 }
```

02

Coroutine builder function

CoroutineScope.async(

)

```
1 suspend fun main() = withContext(Dispatchers.Default) { // this: CoroutineScope
2     // Starts downloading the first page
3     val firstPage = this.async {
4         delay(50.milliseconds)
5         "First page"
6     }
7
8     // Starts downloading the second page in parallel
9     val secondPage = this.async {
10        delay(100.milliseconds)
11        "Second page"
12    }
13
14    // Awaits both results and compares them
15    val pagesAreEqual = firstPage.await() == secondPage.await()
16    println("Pages are equal: $pagesAreEqual")
17 }
```

Hình 9: Ví dụ về CoroutineScope.async()

Coroutine dispatchers

01

Chỉ định Dispatcher cho Coroutine

Để chỉ định dispatcher rõ ràng, bạn có thể truyền nó trực tiếp vào coroutine builder

```
1 suspend fun main() = coroutineScope {  
2     launch(Dispatchers.Default) {  
3         println("Running on ${Thread.currentThread().name}")  
4     }  
5 }
```

Hình 11: Ví dụ về Dispatcher cho Coroutine

Coroutine dispatchers

02

Chuyển Dispatcher bằng withContext()

Trong nhiều trường hợp, ta không muốn tạo coroutine mới mà chỉ muốn chuyển toàn bộ đoạn code sang một dispatcher khác. Khi đó, ta dùng withContext().

```
1 suspend fun main() {  
2     withContext(Dispatchers.IO) {  
3         println("IO work on ${Thread.currentThread().name}")  
4     }  
5  
6     println("Back to original dispatcher: ${Thread.currentThread().name}")  
7 }
```

Hình 12: Ví dụ về chuyển Dispatcher bằng

Coroutine dispatchers

03

Các Dispatcher phổ biến

- **Dispatchers.Default:** Dùng cho các tác vụ nặng CPU như xử lý dữ liệu, tính toán.
- **Dispatchers.IO:** Dùng cho các tác vụ I/O như gọi mạng, đọc ghi file, truy cập database.
- **Dispatchers.Main:** Chạy trên UI thread, dùng để cập nhật giao diện
- **Dispatchers.Unconfined:** Không đảm bảo coroutine resume trên thread nào, chỉ phù hợp cho coroutine rất nhẹ.
- **newSingleThreadContext:** Tạo một thread riêng cho coroutine, tốn tài nguyên và cần quản lý cẩn thận.

Cancellation



- User mở màn hình -> Loading....
- User ấn Back/Home -> App vẫn chạy ngầm?
- Hậu quả:
 - Tốn CPU & Pin.
 - Crash UI.



Cancellation sinh ra để dừng việc không cần thiết

Cancellation

01

Cơ chế hoạt động

Trong Kotlin, mọi coroutine đều gắn với một Job.

- `launch()` → trả về Job.
- `async()` → trả về Deferred (một dạng Job).

→ Job đại diện cho vòng đời sống - chết của coroutine.

```
1 fun main() = runBlocking {
2     println("--- Start ---")
3
4     // 1. Init Job
5     val job = launch {
6         println("Job running...")
7         delay(1000)
8         println("Job Done")
9     }
10
11    // 2. Wait & Cancel
12    delay(300)
13    println("Cancelling job...")
14    job.cancel()
15    job.join()
16
17    println("--- End ---")
18 }
```

Hình 13: Cơ chế hoạt động của

Cancellation

01

Cơ chế hoạt động

```
--- Start ---  
Job running...  
Cancelling job...  
--- End ---
```

- Ở đây: `job.cancel()` không giết coroutine.
- Nó chỉ gửi tín hiệu yêu cầu dừng.

Hình 14: Kết quả khi chạy cơ chế hoạt động của Coroutine

Cơ chế hoạt động

```
1 fun main() = runBlocking {
2     val parent = launch {
3         repeat(2) { index ->
4             launch {
5                 try {
6                     println("[Child $index] Đang chạy...")
7                     awaitCancellation()
8                 } finally {
9                     println("[Child $index] Đã bị hủy!")
10                }
11            }
12        }
13    }
14
15    delay(200)
16    println(">>> Gọi: parent.cancel()")
17    parent.cancelAndJoin()
18    println("--- Kết thúc ---")
19 }
```

Hình 15: Nguyên tắc Structured Concurrency

- Áp dụng nguyên tắc
Structured Concurrency: Khi
coroutine cha bị hủy → tất cả
coroutine con bị hủy theo.

```
[Child 0] Đang chạy...
[Child 1] Đang chạy...
>>> Gọi: parent.cancel()
[Child 0] Đã bị hủy!
[Child 1] Đã bị hủy!
--- Kết thúc ---
```

Hình 16: Kết quả

Cancellation

02.1

Coroutine phản ứng như thế nào ?

```
1 fun main() = runBlocking {  
2     val job = launch(Dispatchers.Default) {  
3         while (true) {  
4             println("Worker: Đang làm việc...")  
5             Thread.sleep(100)  
6         }  
7     }  
8     delay(250)  
9     println("Manager: Sa thải!")  
10    job.cancel()  
11    delay(300)  
12 }
```

Hình 17: Coroutine không tự dừng nếu không hợp tác

```
Worker: Đang làm việc...  
Worker: Đang làm việc...  
Manager: Sa thải!  
Worker: Đang làm việc...  
Worker: Đang làm việc...
```

Hình 18: Kết quả

Cancellation

02.2

Các cách chính để Coroutine hợp tác

Dùng suspend function có sẵn: `delay()`, `await()`, `yield()`

Các hàm này:

- Tự động kiểm tra `Job.isActive`
- Nếu đã bị hủy → ném `CancellationException`
- Coroutine kết thúc ngay

VD: `delay(1000)` → Nếu bị `cancel`, sẽ kết thúc tại đây.

Cancellation

02.2

Các cách chính để Coroutine hợp tác

Kiểm tra thủ công (khi không có suspend)

Nếu coroutine chạy:

- vòng lặp tính toán nặng
- thuật toán xử lý ảnh
- không có suspend point

→ bắt buộc phải tự kiểm tra.

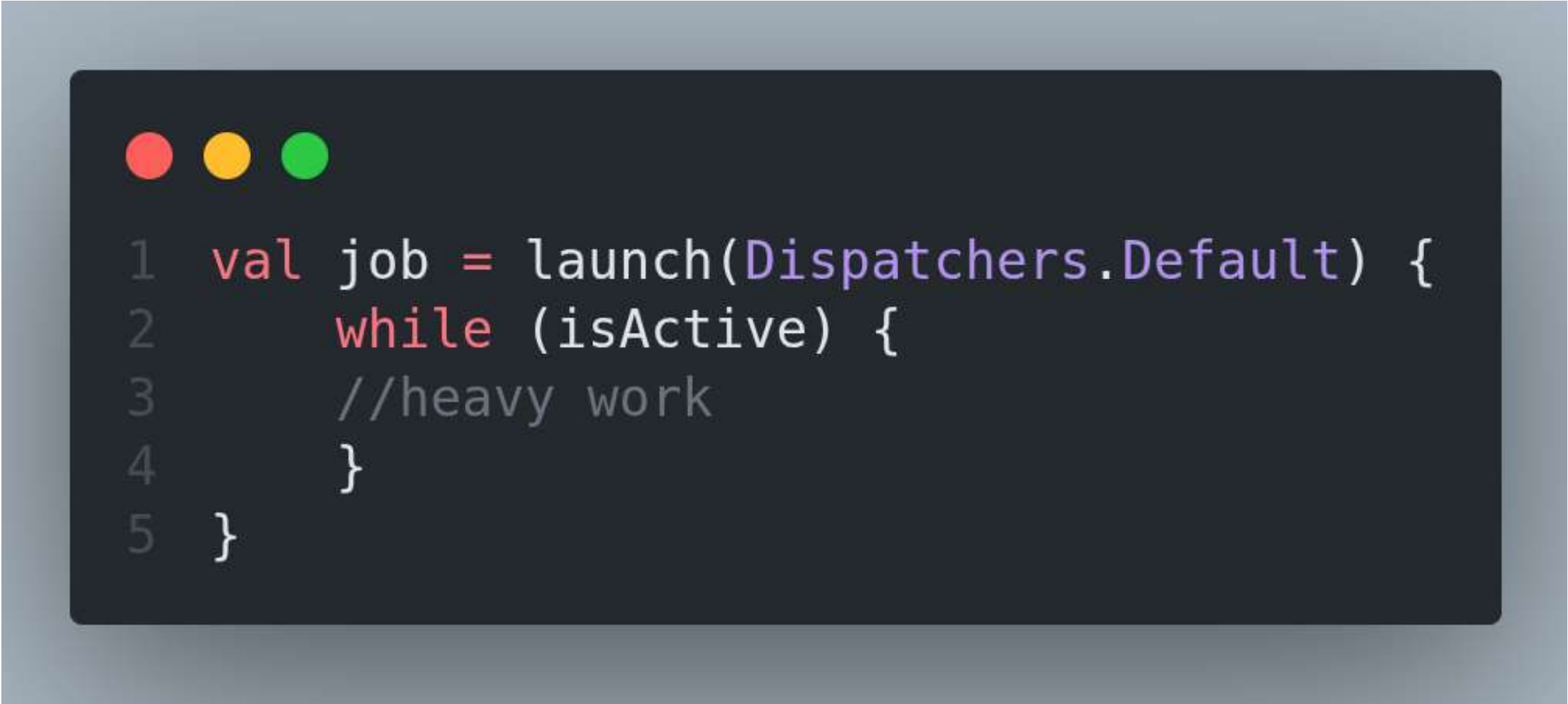
Cancellation

02.2

Các cách chính để Coroutine hợp tác

Dùng isActive:

- isActive chỉ kiểm tra chờ
- Rất nhanh, rất rẻ
- Phù hợp cho vòng lặp



```
1 val job = launch(Dispatchers.Default) {  
2     while (isActive) {  
3         //heavy work  
4     }  
5 }
```

Hình 19: Dùng isActive để Coroutine hợp tác

Cancellation

02.2

Các cách chính để Coroutine hợp tác

Dùng `ensureActive()`

- Nếu bị hủy → ném `CancellationException` ngay
- Dùng khi muốn fail-fast

Cancellation

03

Xử lý tài nguyên & NonCancellable

Cancellation có thể xảy ra ở bất kỳ suspension point nào.

Nghĩa là:

- coroutine có thể chết giữa chừng
- không theo ý mình
- không có cơ hội chạy code phía sau

Vì vậy, cleanup không được đặt ở code thường.

Cancellation

Xử lý tài nguyên & Non Cancellable

```
1 fun main() = runBlocking {
2     val job = launch {
3         try {
4             awaitCancellation()
5         } finally {
6             withContext(NonCancellable)
7         }
8         {
9             delay(1000)
10            println("Cleanup Done!")
11        }
12    }
13    delay(200)
14    job.cancelAndJoin()
15 }
```

Hình 20: Giải pháp
try/finally

Cancellation

03

Xử lý tài nguyên & Non Cancellable

- Cảnh báo: “NonCancellable là dao hai lưỡi. Nếu bên trong chạy network treo, IO dài, hoặc logic phức tạp → coroutine sẽ không bao giờ chết, gây memory leak.”
- Nguyên tắc: Chỉ dùng NonCancellable cho cleanup, cực kỳ ngắn, nên có timeout riêng.

Cancellation

04

Timeout

```
1 fun main() = runBlocking {  
2     val result = withTimeoutOrNull(100)  
3     {  
4         delay(300)  
5         "Success"  
6     }  
7     println("Kết quả: $result")  
8 }
```

Kết quả: Hàm tự động hủy và trả về null nhẹ nhàng thay vì ném Exception làm crash ứng dụng. Code cực kỳ an toàn.

Hình 21: Timeout

Exception Handling

01

Lan truyền lỗi: Launch và Async

- launch: coi lỗi là 'uncaught exception'. Nghĩa là ném lỗi cái là crash ngay lập tức.
- async: 'giấu' lỗi vào trong đối tượng kết quả (Deferred). Lỗi sẽ không nổ ra cho đến khi gọi hàm `.await()`.

→ Vì vậy: Với launch phải bắt lỗi ngay, còn async thì phải try-catch chỗ `await`."

Exception Handling

01

Lan truyền lỗi: Launch và Async

```
1 fun main() = runBlocking {  
2     supervisorScope {  
3         val d = async {  
4             println("async starts")  
5             delay(100)  
6             throw RuntimeException("boom from async")  
7         }  
8  
9         delay(50)  
10        println("Before await")
```

```
1         try {  
2             d.await()  
3         } catch (e: Exception) {  
4             println("Caught at await: $e")  
5         }  
6  
7         println("End main")  
8     }  
9 }
```

```
Before await  
Caught at await: java.lang.RuntimeException: boom from async  
End main
```

Hình 22: Ví dụ về Async và kết quả khi chạy

Exception Handling

02

CoroutineExceptionHandler (CEH) & Aggregation

Để bắt các lỗi không mong muốn ở launch, ta dùng

CoroutineExceptionHandler

- Nó chỉ hoạt động trên Root Coroutine.
- Aggregation (Gộp lỗi): Nếu nhiều coroutine con cùng lỗi một lúc, quy tắc là 'Ai lỗi trước người đó thắng'.

Exception Handling

03

Supervision

Normal Job: Hủy lan truyền 2 chiều.

SupervisorJob: Hủy chỉ đi 1 chiều từ trên xuống. Nếu con bị lỗi, cha không bị ảnh hưởng, các anh em khác vẫn sống khỏe.

Exception Handling

03

Supervision

```
1 fun main() = runBlocking {
2     supervisorScope {
3         launch {
4             try {
5                 throw RuntimeException("fail here")
6             } catch (e: Exception) {
7                 println("Child handled its own error: $e")
8             }
9     }
```

```
1         launch {
2             delay(200)
3             println("Other child still runs")
4         }
5     }
6 }
```

```
Exception in thread "main" java.lang.IllegalArgumentException: No main method found in project
at com.compiler.server.compiler.components.KotlinCompiler.run$lambda$0 (KotlinCompiler.kt:67)
at com.compiler.server.compiler.components.KotlinCompiler.execute$lambda$0 (KotlinCompiler.kt:148)
at com.compiler.server.compiler.components.CliUtilsKt.usingTempDirectory (CliUtils.kt:119)
```

Hình 23: Ví dụ về Supervision và kết quả khi chạy

Asynchronous Flow

01

Khác biệt với Sequence

Sequence trả về tất cả các giá trị cùng 1 lúc, nghĩa là tất cả các hoạt động trong Sequence đều phải hoàn thành thì hàm print mới chạy

```
1 suspend fun simple(): List<Int> {  
2     delay(1000) // pretend we are doing something asynchronous here  
3     return listOf(1, 2, 3)  
4 }  
5  
6 fun main() = runBlocking<Unit> {  
7     simple().forEach { value -> println(value) }  
8 }
```

Asynchronous Flow

01

Khác biệt với Sequence

Ngược lại, Flow sẽ trả về các giá trị theo tuần tự mà không phải chờ tất cả dữ liệu được tính toán xong

```
1 fun simple(): Flow<Int> = flow { // flow builder
2     for (i in 1..3) {
3         delay(100) // pretend we are doing something useful here
4         emit(i) // emit next value
5     }
6 }
7
8 fun main() = runBlocking<Unit> {
9     // Launch a concurrent coroutine to check if the main thread is blocked
10    launch {
11        for (k in 1..3) {
12            println("I'm not blocked $k")
13            delay(100)
14        }
15    }
16    // Collect the flow
17    simple().collect { value -> println(value) }
18 }
```

Asynchronous Flow

01

Khác biệt với Sequence



```
1 I'm not blocked 1
2 1
3 I'm not blocked 2
4 2
5 I'm not blocked 3
6 3
```

Hình 26: Kết quả chạy của từng số mỗi 100ms

Asynchronous Flow

02

Đặc tính “Cold Stream” của Flow



```
1 fun simple(): Flow<Int> = flow {  
2     println("Flow started")  
3     for (i in 1..3) {  
4         delay(100)  
5         emit(i)  
6     }  
7 }
```

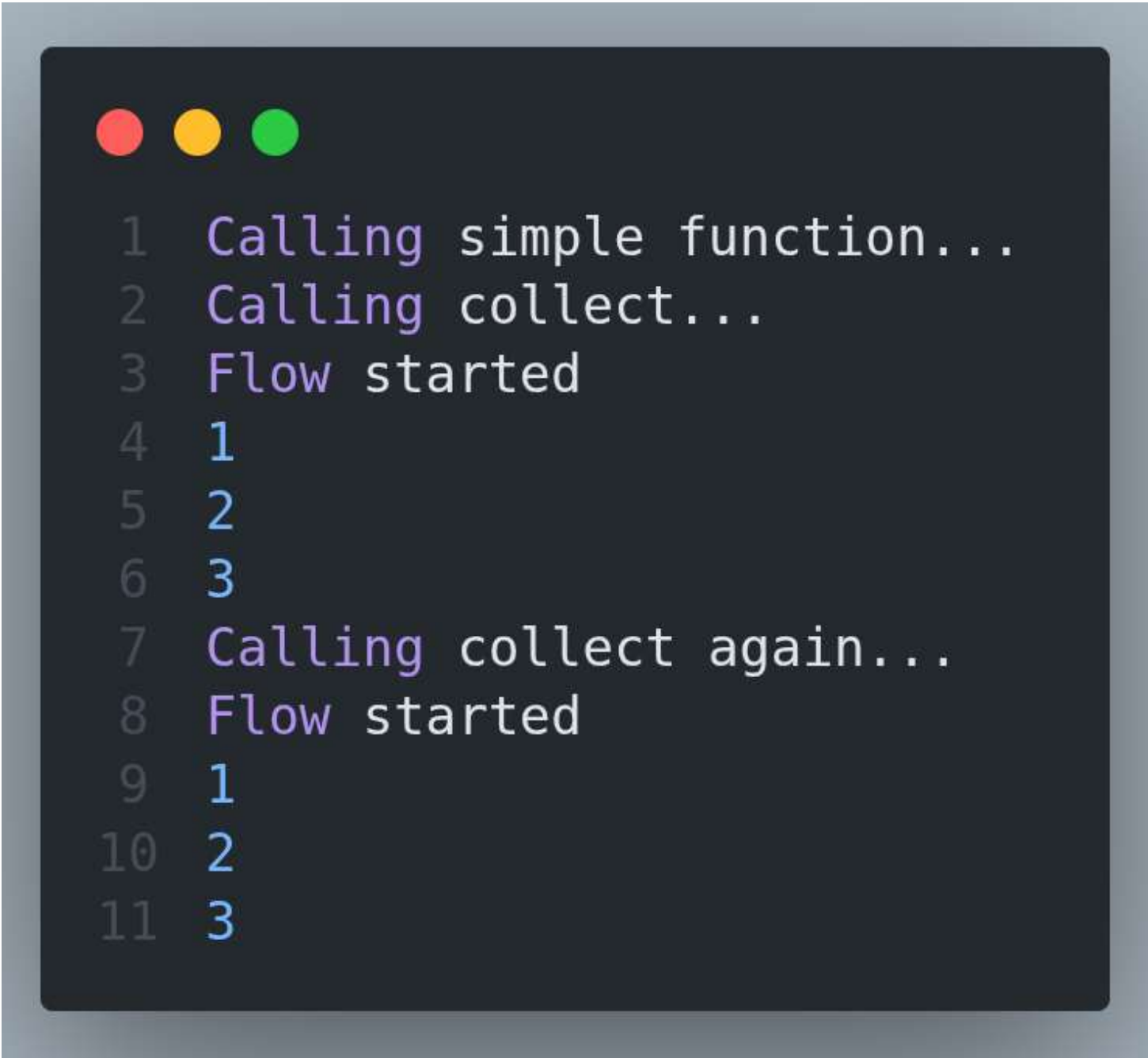


```
1 fun main() = runBlocking<Unit> {  
2     println("Calling simple function...")  
3     val flow = simple()  
4     println("Calling collect...")  
5     flow.collect { value -> println(value) }  
6     println("Calling collect again...")  
7     flow.collect { value -> println(value) }  
8 }
```

Asynchronous Flow

02

Đặc tính “Cold Stream” của Flow



```
1 Calling simple function...
2 Calling collect...
3 Flow started
4 1
5 2
6 3
7 Calling collect again...
8 Flow started
9 1
10 2
11 3
```

Hình 28: Kết quả khi chạy Cold Stream

Asynchronous Flow

03

Tính tuần tự của Flow

```
1 (1..5).asFlow()
2   .filter {
3       println("Filter $it")
4       it % 2 == 0
5   }
6   .map {
7       println("Map $it")
8       "string $it"
9   }
10  .collect {
11      println("Collect $it")
12  }
```

Hình 29: Lọc các số nguyên chẵn và ánh xạ chúng thành chuỗi

Asynchronous Flow

03

Tính tuần tự của Flow

Quan sát kết quả, ta thấy tính tuần tự:

- Filter 1 chạy. $1 \% 2 \neq 0$ nên không chạy Map.
- Filter 2 chạy. $2 \% 2 == 0$ nên chạy Map 2.
- Sau khi Map 2 hoàn thành, chạy Collect string 2.
- Lặp lại cho Filter 3 (bỏ qua Map và Collect).
- Filter 4 -> Map 4 -> Collect string 4.
- Filter 5 (bỏ qua Map và Collect).

```
Filter 1
Filter 2
Map 2
Collect string 2
Filter 3
Filter 4
Map 4
Collect string 4
Filter 5
```

Hình 30: Kết quả

Asynchronous Flow

04

Xây dựng 1 Flow

- **Sử dụng flow { ... }:** Đây là builder cơ bản nhất, cho phép bạn sử dụng hàm emit() để phát ra các giá trị từ bên trong một suspend lambda.

```
1 fun simple(): Flow<Int> = flow { // Flow builder
2     for (i in 1..3) {
3         emit(i) // Emit next value
4     }
5 }
```

Hình 31: Xây dựng Flow sử dụng flow { ... }

Asynchronous Flow

04

Xây dựng 1 Flow

- **Sử dụng `flowOf()`:** Builder này dùng để định nghĩa một Flow phát ra một tập hợp các giá trị cố định.

```
1 fun simple(): Flow<String> = flowOf("one", "two", "three")
2 simple().collect { value -> println(value) } // prints "one", "two", "three"
```

Hình 32: Xây dựng Flow sử dụng `flowOf()`

Asynchronous Flow

04

Xây dựng 1 Flow

- **Sử dụng `.asFlow()` extension function:** Cho phép chuyển đổi các kiểu dữ liệu đã có như Collection (List, Set) hoặc Range thành Flow.// Chuyển đổi một Range số nguyên thành một Flow

```
1 (1..3).asFlow().collect {value -> println(value) }  
2 // Prints: 1, 2, 3  
3 // Chuyển đổi một lượng List thành một Flow  
4 listOf("A", "B", "C").asFlow().collect {value -> println(value) }  
5 // Prints: A, B, C
```

Asynchronous Flow

05

Toán tử trung gian (Intermediate Operators)

```
1 suspend fun performRequest(request: Int): String {  
2     delay(1000) // imitate long-running asynchronous work  
3     return "response $request"  
4 }  
5  
6 fun main() = runBlocking<Unit> {  
7     (1..3).asFlow() // a flow of requests  
8     .map { request -> performRequest(request) }  
9     .collect { response -> println(response) }  
10 }
```

Hình 34: Ví dụ về toán tử trung gian

Asynchronous Flow

05.1

Toán tử Transform

sử dụng transform chúng ta có thể phát ra một chuỗi trước khi thực hiện một yêu cầu bất đồng bộ chạy dài và theo sau là phản hồi

```
1 (1..3).asFlow()
2   .transform { request ->
3       emit("Making request $request")
4       emit(performRequest(request))
5   }
6   .collect { response -> println(response) }
```

Asynchronous Flow

05.2

Toán tử giới hạn kích thước (Size-limiting operators)

- Các toán tử trung gian giới hạn kích thước như `take` sẽ hủy việc thực thi của flow khi giới hạn tương ứng đạt được.
- Việc hủy trong coroutine luôn được thực hiện bằng cách ném ra một ngoại lệ, để tất cả các hàm quản lý tài nguyên (như khối `try { ... } finally { ... }`) hoạt động bình thường trong trường hợp hủy.

Asynchronous Flow

05.2

Toán tử giới hạn kích thước (Size-limiting operators)

```
1 fun numbers(): Flow<Int> = flow {
2     try {
3         emit(1)
4         emit(2)
5         println("This line will not execute")
6         emit(3)
7     } finally {
8         println("Finally in numbers")
9     }
10 }
11
12 fun main() = runBlocking<Unit> {
13     numbers()
14         .take(2) // chỉ lấy hai giá trị đầu tiên
15         .collect { value -> println(value) }
16 }
```

Hình 36: Ví dụ về toán tử giới hạn kích thước



fit@hcmus



THANK YOU FOR LISTENING!

